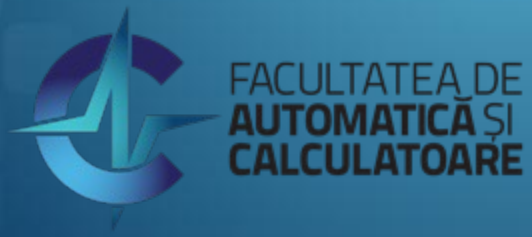


Algoritmi Paraleli și Distribuiți
Terminarea programelor distribuite

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro



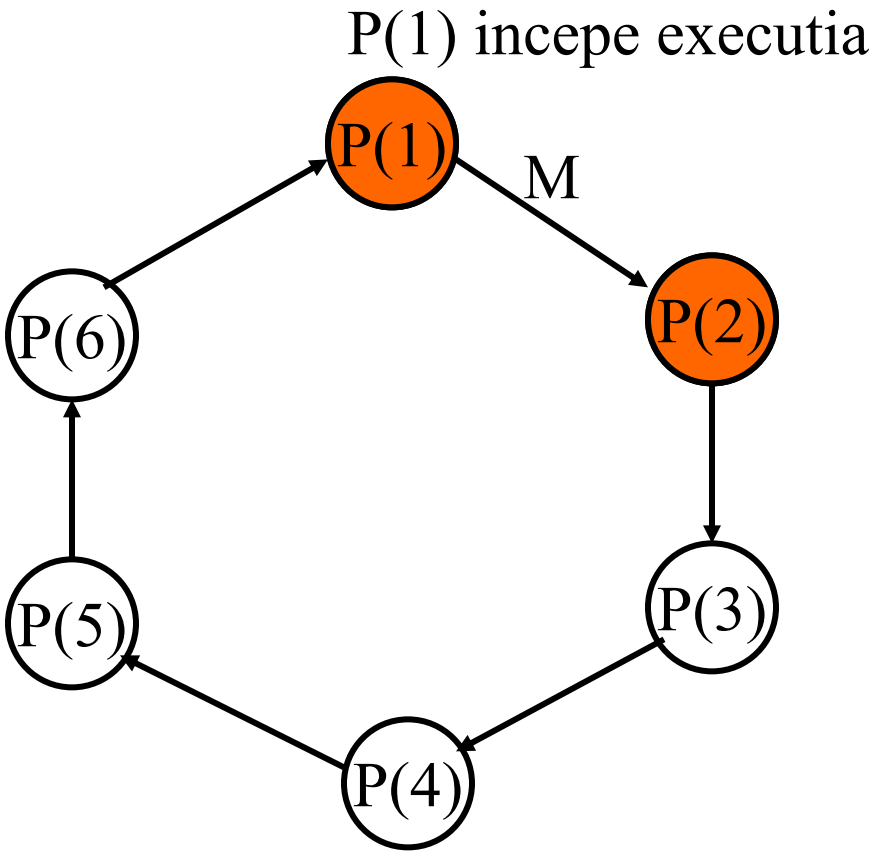


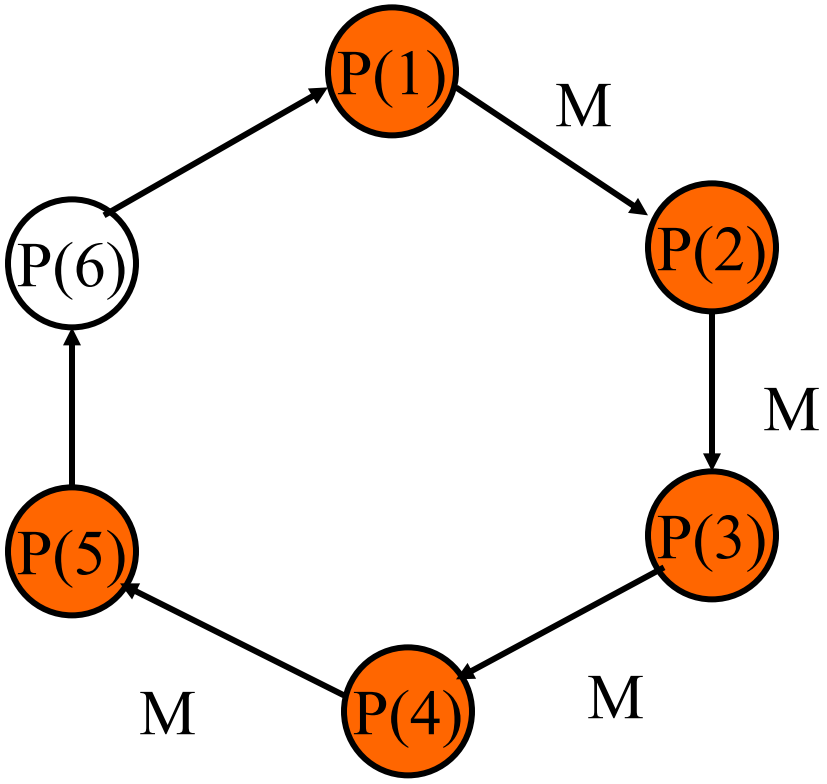
Terminarea programelor distribuite

- **Problema:** detecția terminării
- Algoritmi paraleli:
  - toate procesele se termină
    - cicluri infinite?
  - procese terminate sau blocate și nu există I/E în curs
  - simplu pentru un singur procesor - coada *ready* goală

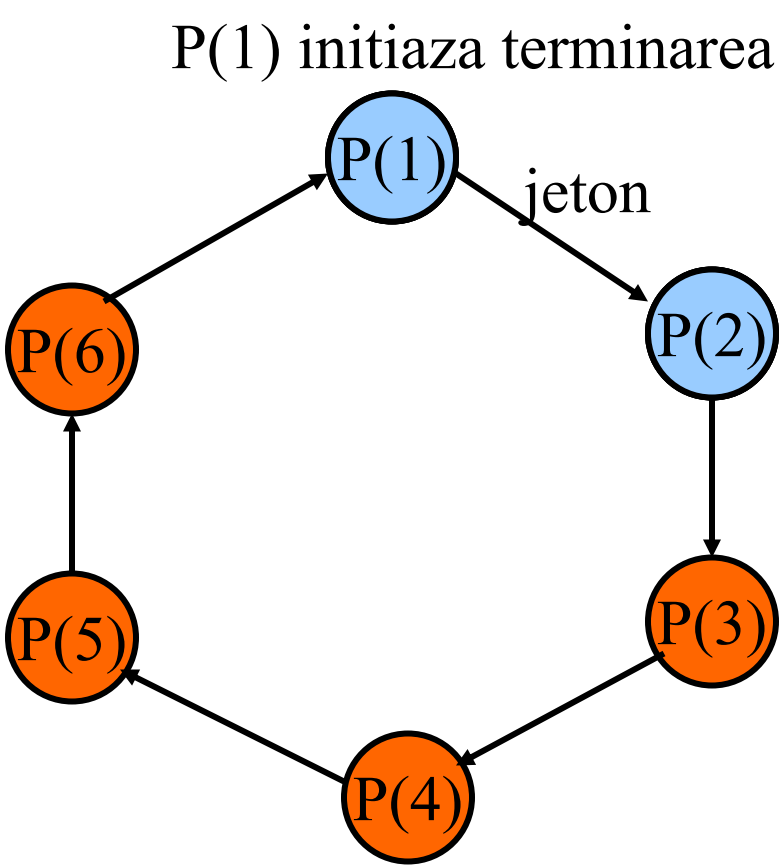
- Algoritmi distribuiți:
  - fiecare proces să fie liber
    - a terminat execuția *sau*
    - se află în aşteptarea unui mesaj (**receive**)
  - pe canale să nu existe mesaje în tranzit

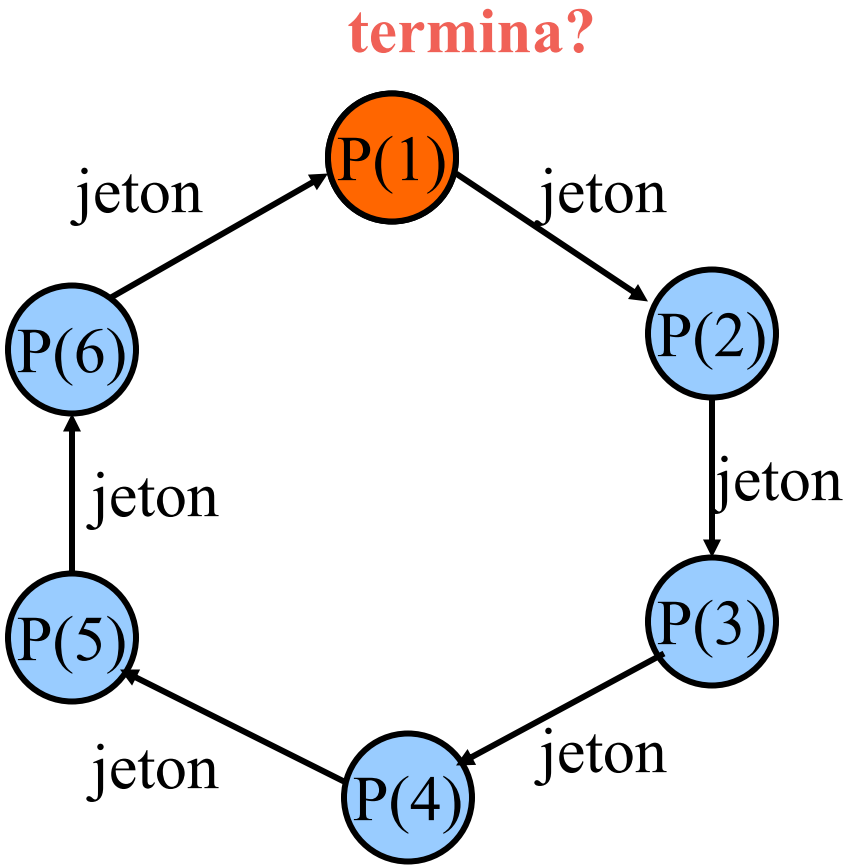
Procese organizate în inel





Procese organizate în inel





inainte de a primi jeton, P(1) poate fi reactivat de un mesaj de la P(6)!

Procese organizate în inel (tehnica jetoanelor) (1)

- Inițiatorul terminării este procesul P[1]
- acțiunile lui P[1] cînd devine liber prima dată:

```
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

- acțiunile lui P[i=1 to N] la recepția unui mesaj obişnuit:

```
culoare[i] = roșu;
```

Tehnica jetoanelor (2)

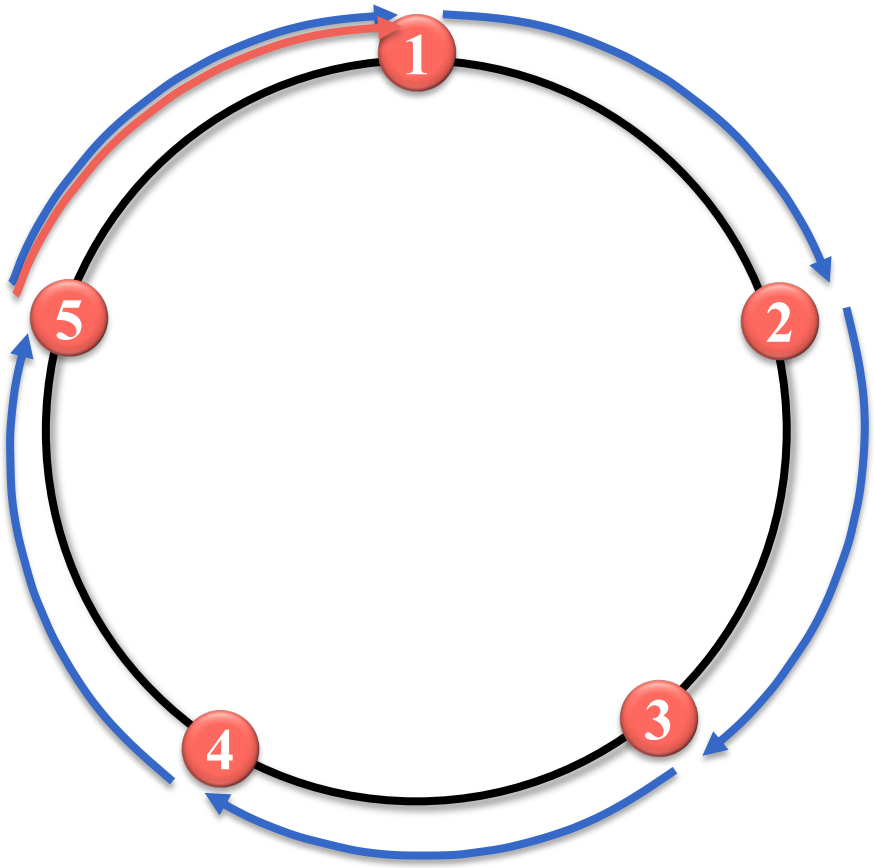
- acțiunile lui P[i=2 to N] la primirea unui jeton (și după ce devine liber):

```
culoare[i] = albastru;
jeton = jeton +1;    /* nesemnificativ */
send ch[(i + 1) mod n](jeton);
```

- acțiunile lui P(1) la recepția jetonului:

```
if (culoare[1] == albastru) {
  anunță terminarea; stop
}
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

Tehnica jetoanelor (3)

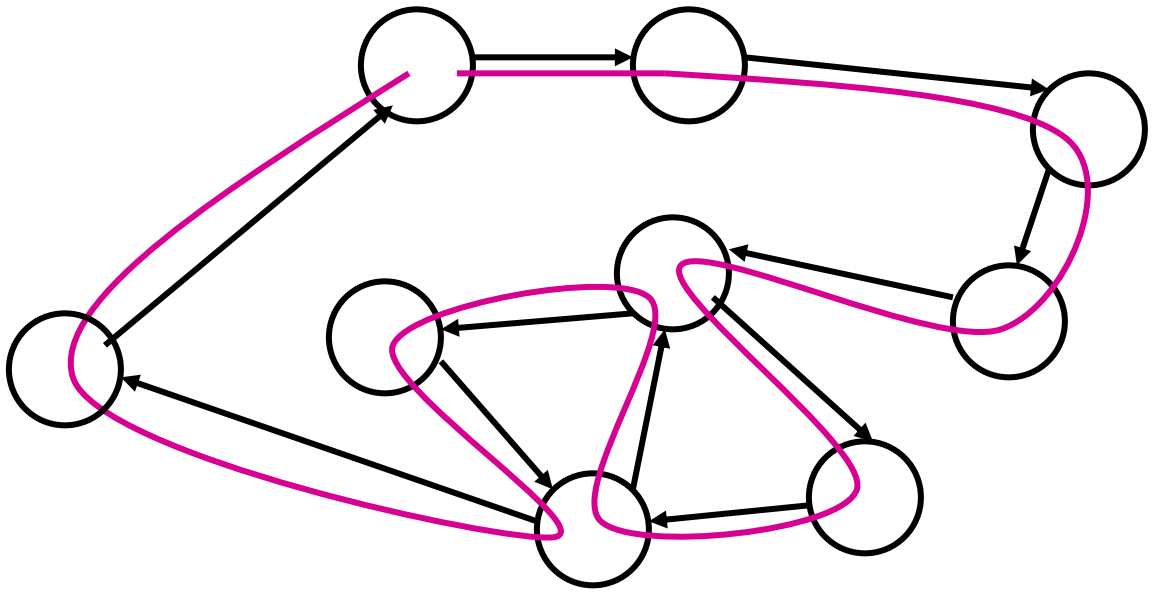


Jeton
1

STOP

Terminarea în cazul general (tehnica jetoanelor)(1)

- topologie generală de **graf**
- **c** un ciclu care include toate arcele grafului; fie **nc** lungimea lui



Terminarea în cazul general (tehnica jetoanelor)(2)

- un jeton care se transmite după mesajele de date
- are inițial valoarea 0
- **acțiunile lui P[i=1 to N] la primirea unui mesaj uzual:**

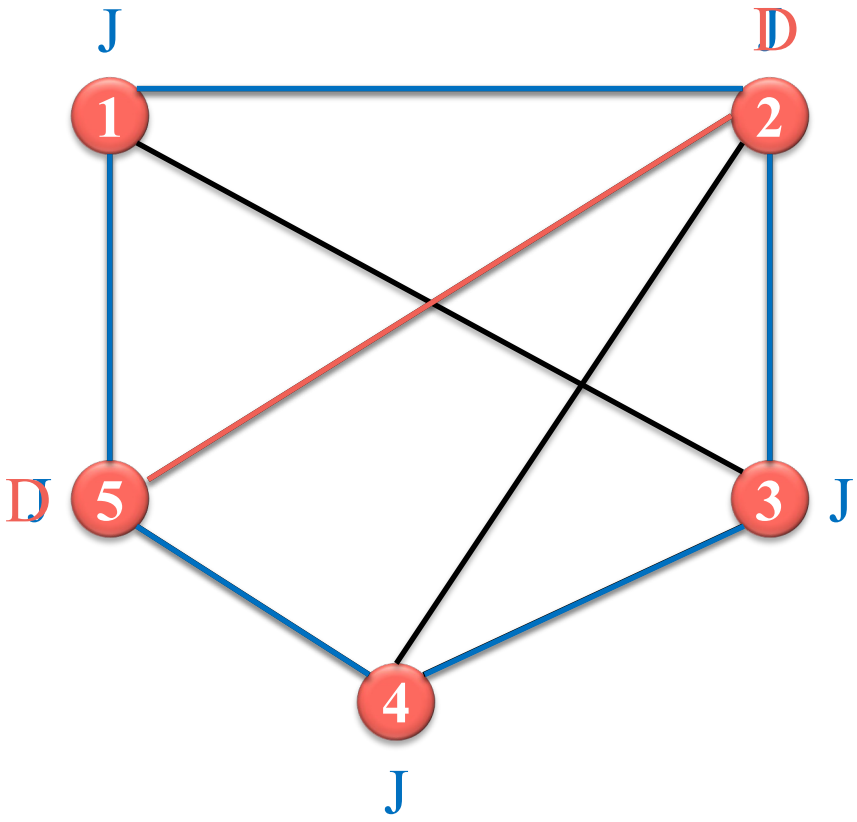
```
culoare[i] = roșu;
```

- **acțiunile lui P[i=1 to N] la recepția jetonului:**

```
if (jeton == nc) { anunță terminarea; stop }
if (culoare[i] == roșu) { culoare[i] = albastru;
                          jeton = 0; }
else if (culoare[i] == albastru) jeton = jeton +1;

setează j la canalul următor din ciclul c;
send ch[j](jeton);
```

Terminarea în cazul general (tehnica jetoanelor)(3)



nc = 5

Jeton
0

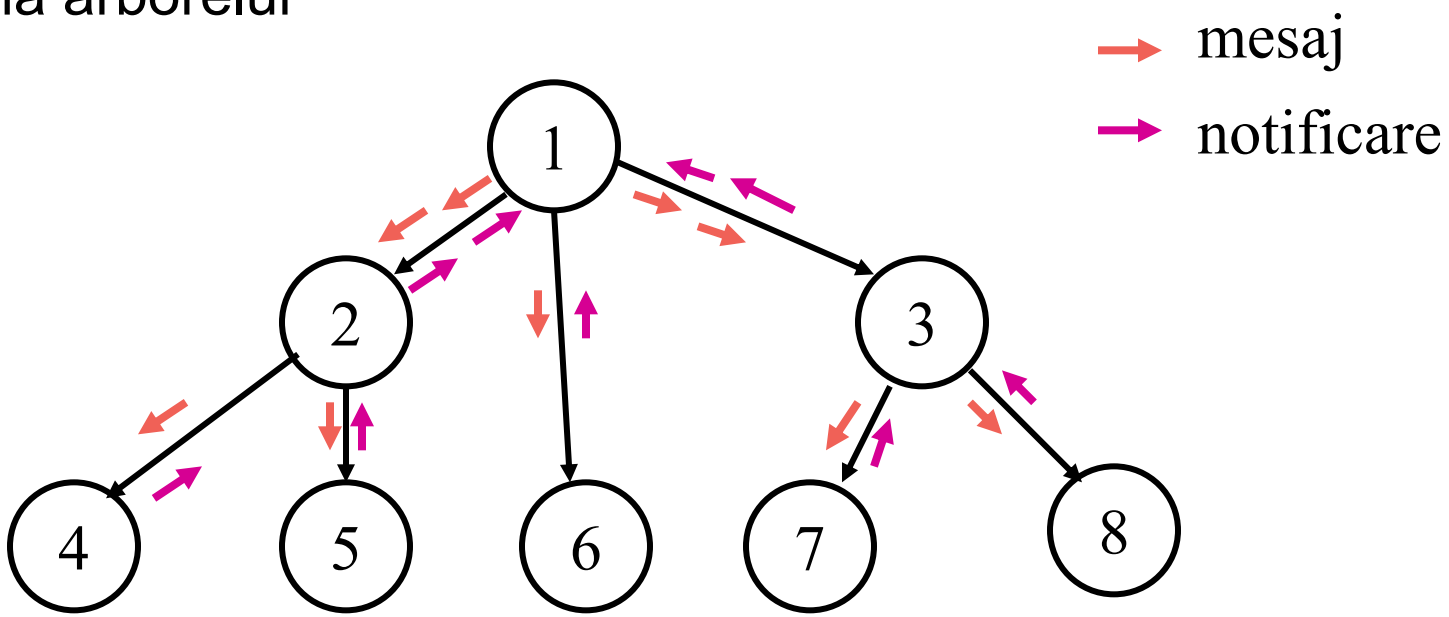
STOP

Terminare (topologii arbore)

- Tehnica de confirmare a mesajelor -

Terminarea in topologii arbore

- Când un proces frunză se termină, el anunţă părintele său
- Când un nod oarecare al arborelui se termină, el aşteaptă notificarea terminării de la toţi copiii săi şi apoi îşi anunţă părintele
- Notificările se propagă astfel pînă la procesul sursă, aflat în rădăcina arborelui



Confirmarea mesajelor (1)

- Funcția de determinare a stării canalelor se poate împărți între procesele programului: fiecare proces verifică doar canalele pe care a trimis mesaje de date altor procese (canalele sale de ieşire)

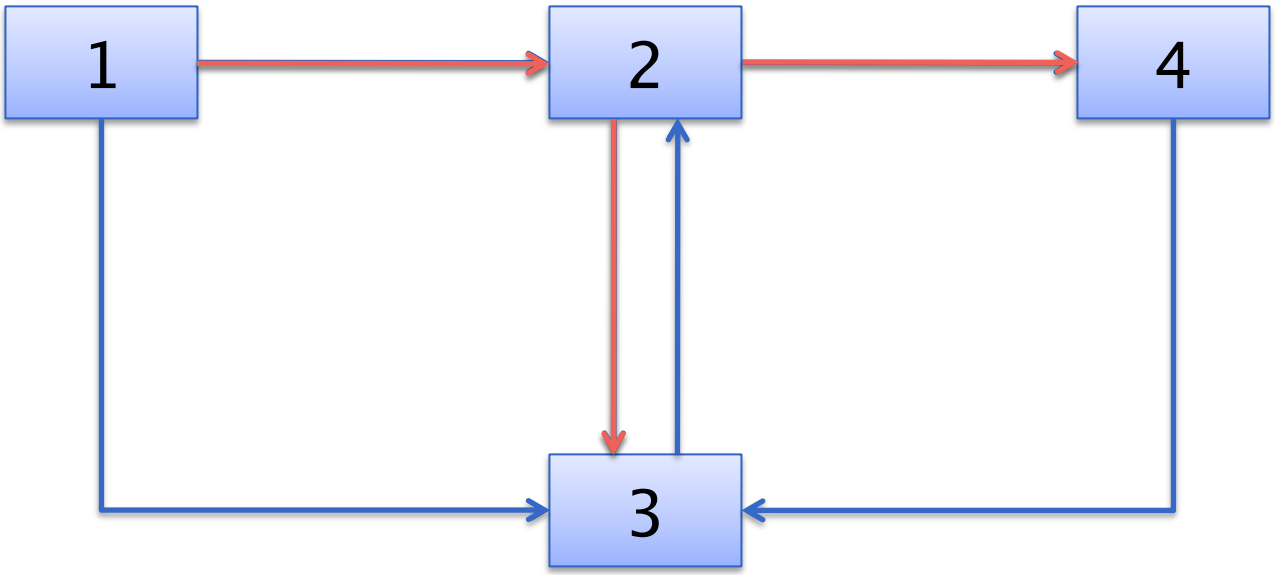
- Pe un canal
  - se trimit **mesaje de date** și se primesc **semnale de confirmare** *sau*
  - se primesc **mesaje de date** și se trimit **semnale de confirmare**

- **Confirmarea mesajelor**: dacă pentru fiecare mesaj de date transmis pe un canal procesul primeşte de la receptor un semnal de confirmare, atunci canalul respectiv este gol

- Presupune existența unui proces **sursă**:
  - care nu are legături de intrare *şi*
  - poate accesa orice alt proces pe o cale oarecare din graf

Confirmarea mesajelor (2)

- Exemplu:
  - Procesul sursă: 1
  - Arbore de acoperire



Algoritmul Dijkstra-Scholten (1)

Volume 11, number 1 INFORMATION PROCESSING LETTERS 29 August 1980

TERMINATION DETECTION FOR DIFFUSING COMPUTATIONS *

Edsger W. DIJKSTRA
Burroughs, 5671 AL Nuenen, The Netherlands

C.S. SCHOLTEN
Philips Research Laboratories, 5656 AA Eindhoven, The Netherlands

Received 2 June 1980

Concurrency, termination detection, distributed control, separation of concerns, message-based systems, correctness proving, diffusing computations, networks, cornets, activation tree, nondeterminacy

The following seems to capture the quintessence of a situation that is not unusual in distributed processing. Consider a finite, directed graph. If the graph contains an edge from node A to node B, we call B 'a successor of A', and A 'a predecessor of B'. We assume the existence of a node without incoming edges; this node will be called 'the environment' (because it acts as such with respect to the rest of the graph). The other nodes will be called 'the internal nodes'.

For each node its initial state will be called 'the neutral state'. A so-called 'diffusing computation' is started when the environment sends – of its own accord, so to speak – a message to one or more of its successors; it is supposed to do this just once. After reception of its first message, an internal node is free to send messages to its successors. It is this feature proper – such that, when the diffusing computation proper has thus terminated, the fact of this completion will eventually be signalled back to the environment. Besides a node's ability to receive messages from its predecessors and to send messages to its successors, we assume each node also to be able to receive 'signals' from its successors and to send 'signals' to its predecessors; in other words, each edge is assumed to be able to accommodate two-way traffic, but only messages of the computation proper in the one direction and signals in the opposite direction. We shall impose that in the total computation – i.e. from the moment that the environment sent its messages to the rest of the graph until it has received the completion signal – each edge will have carried as many messages in the one direction as it has carried signals in the op-



Dijkstra, Edsger W., and Carel S. Scholten. "Termination detection for diffusing computations." *Information Processing Letters* 11.1 (1980): 1-4.

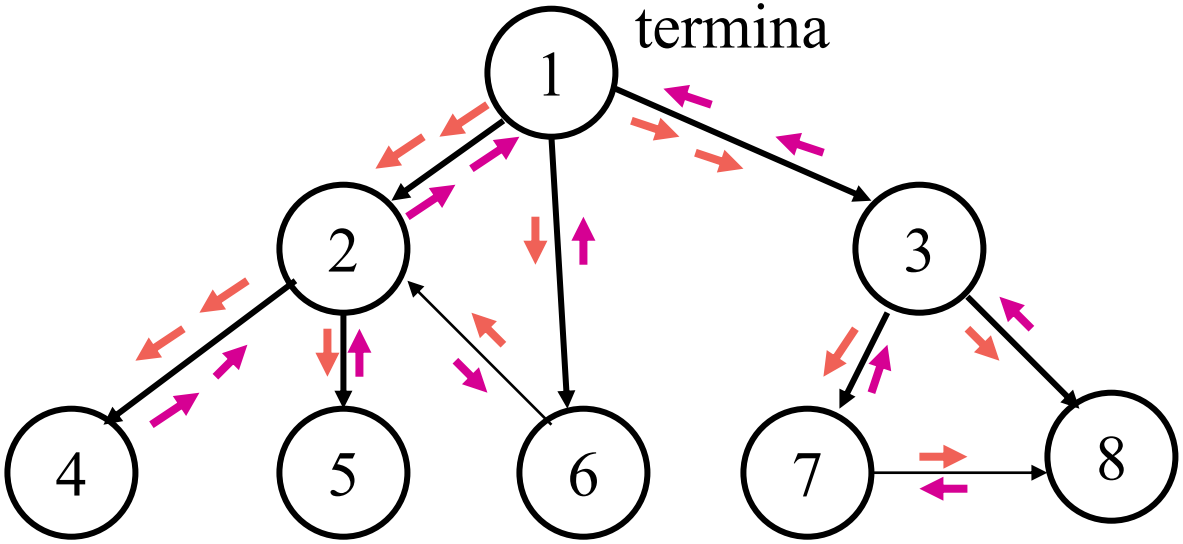
Algoritmul Dijkstra-Scholten (1)

- Bazat pe transmiterea unor semnale de confirmare
- Cazul unei rețele cu topologie **arborescentă**:
  - Când un proces *frunză* se termină, el îl anunță pe *părintele* său
  - Când un *nod* oarecare al arborelui termină, el aşteaptă notificarea terminării de la toți *copiii* săi și apoi își anunță *părintele*
  - Notificările se propagă până la procesul sursă
- Cazul unui **graf dirijat aciclic** de procese
  - Deficitul unei legături = diferența dintre numărul de mesaje de date transmise şi numărul de semnale de confirmare primite pe acea legătură
  - Când un nod doreşte să termine:
    - aşteaptă primirea unor semnale pe legăturile de ieşire a datelor, semnale care reduc deficitele la zero (pe acele legături)
    - trimite pe fiecare legătură de intrare a datelor un număr de semnale egal cu deficitul (legăturii respective)

Algoritmul Dijkstra-Scholten (2)

- **Cazul general**: cicluri în graful proceselor
- Rezolvare: generare **arbore de acoperire**:
  - fiecare legătură generată la prima recepție (adică prima legătură de intrare, numită **prim** – de la părinte)
- Pentru terminare:
  - trimitere semnale pentru toate legăturile de intrare, mai puțin **prim**
  - recepție semnale pentru toate legăturile de ieşire
  - trimitere semnale spre **prim**
- Schema de semnalare este separată și suplimentară față de prelucrarea propriu-zisă

Algoritmul Dijkstra-Scholten



Algoritmul Dijkstra-Scholten (3)

LORDUL





SERVITORUL

Algoritmul Dijkstra-Scholten (4)



- Comunică doar cu alți lorzi și cu propriul servitor
- Comunică cu lorzii prin scrisori (prin poștă) și cu servitorul prin semnale de comunicare (la telefon)
- Își anunță servitorul când trimite/primește o scrisoare (spunându-i și destinația/sursa)
- După ce se plictisește întreabă servitorul dacă a terminat treaba ca să poată merge la vânătoare



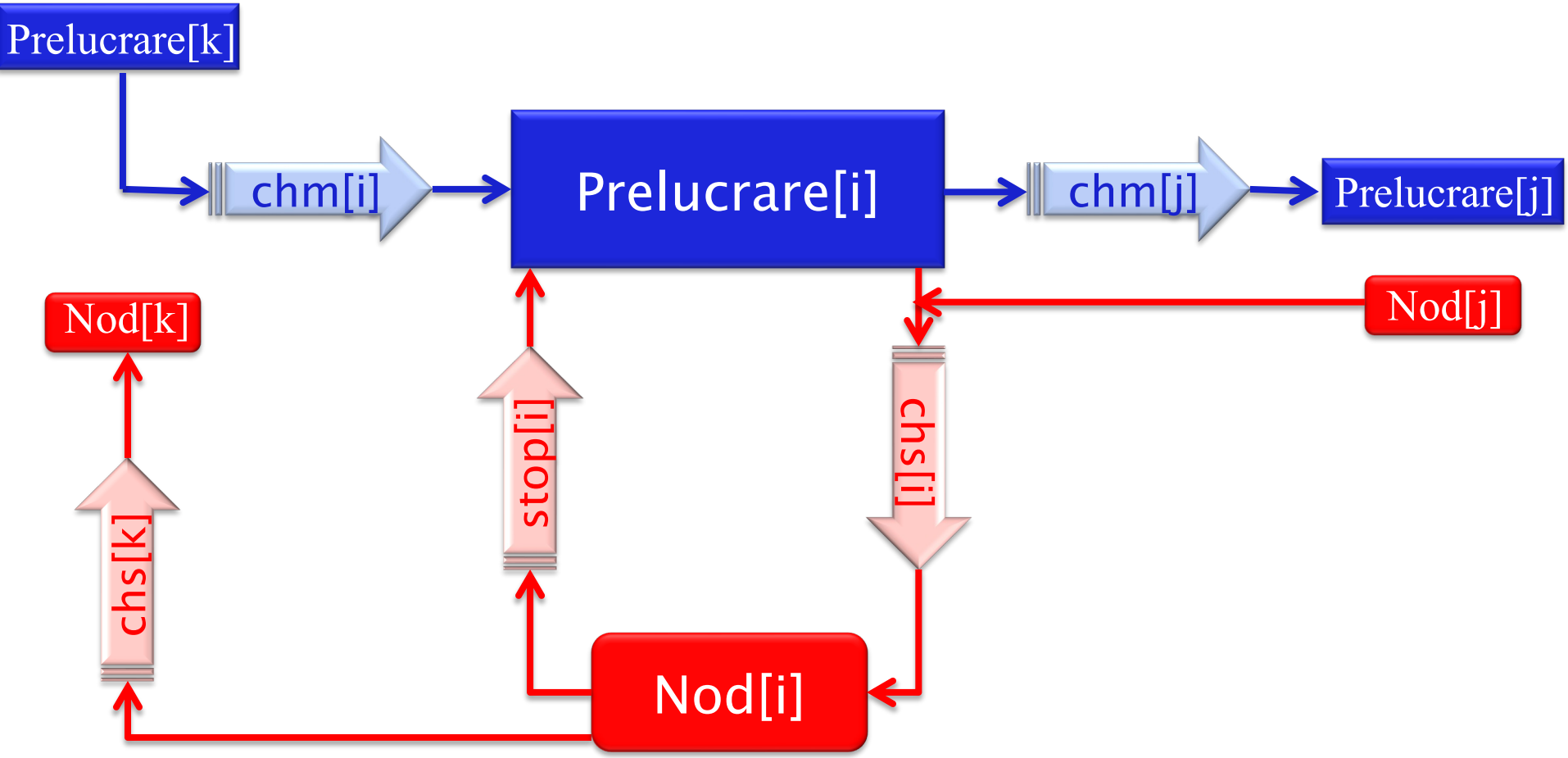
- Comunică doar cu alți servitori și cu propriul lord
- Comunică doar prin semnale (la telefon)
- Trimite semnale de confirmare pentru scrisorile primite și așteaptă semnale de confirmare pentru scrisorile trimise
  - Scrisoarea primită de la lordul$_i$ se confirmă servitorului$_i$
- Răspunde afirmativ doar dacă a primit și trimis toate confirmările pentru mesajele trimise și primite, altfel îl roagă pe lord să aștepte

Algoritmul Dijkstra-Scholten (5)

În continuare

- Lord = Prelucrare *(prelucrează mesaje de date)*
- Servitor = Nod *(gestionează semnalele de confirmare cu alte noduri și semnalele de comunicare cu Prelucrare)*
- Canalul poștal = canalul chm
  - chm_i este „adresa” lui $Prelucrare_i$
  - $Prelucrare_i$ primește mesaje pe chm_i și îi trimite lui $Prelucrare_j$ mesaje pe chm_j
- Canalul telefon pentru Nod = canalul chs
  - chs_i este „numărul de telefon” al lui Nod_i
  - Nod_i primește semnale pe chs_i (de la alte noduri sau de la $Prelucrare_i$) și trimite semnale de confirmare lui Nod_j pe chs_j
- Canalul telefon pentru Prelucrare = canalul stop
  - $stop_i$ este „numărul de telefon” al lui $Prelucrare_i$
  - $Prelucrare_i$ primește semnale de comunicare (da/nu) pe $stop_i$ de la Nod_i

Algoritmul Dijkstra-Scholten (6)



Algoritmul Dijkstra-Scholten (7)

```
enum fel{semnal, transmitere, receptie, terminare};
typedef struct {bool există;
                int deficit} arc;
chan chs[1:N](fel op, int id);
chan chm[1:N](int id, tip_date date);
chan stop[1:N](bool);

process Prelucrare[i=1 to N]{
   tip_date data; int id; bool term;
   inițializări;
   term = false;
   while (NOT term) {
      receive chm[i](id, data);                  /* primește mesaj */
      send chs[i](recepție, id);                 /* semnalează nod - recepție mesaj */
      realizează prelucrare
      ....
      send chm[j](i, data);                      /* trimite mesaj */
      send chs[i](transmitere, j);               /* semnalează nod - transmitere mesaj */
      ...
      if (decizie de terminare) term = true;
   }
   term = false;
   while (NOT term) {
      send chs[i](terminare, i);                 /* anunță nod că a terminat */
      receive stop[i](term);                     /* așteaptă să termine nodul */
   }
   stop;
```

Algoritmul Dijkstra-Scholten (8)

```
process Nod[i=1 to N] {
   arc in[1:N]; /* arcele de intrare */
   int nsemnale = 0;      /* număr semnale așteptate pentru mesajele trimise */
   int prim = 0;
   var id: int; fel k;
   while (true) {
      receive chs[i](k, id);
      if (k == recepție) { if (prim == 0) prim = id fi;
                           in[id].deficit = in[id].deficit + 1; }
      else if (k == transmitere) nsemnale = nsemnale+1;
      else if (k == semnal) nsemnale = nsemnale-1;
      else if (k == terminare){
      for [j = 1 to N st (in[j].există AND (j <> prim))] /* se confirmă mesajele primite */
         while (in[j].deficit > 0) { send chs[j](semnal, i);
                                     in[j].deficit = in[i].deficit - 1;
         }
      }
      if (nsemnale <> 0) send stop[i](false);            /* mai sunt semnale de primit */
      else if (nsemnale == 0) {
         if ((i <> sursa) AND (prim <> 0)) {              /* se confirmă lui prim */
            while (in[prim].deficit > 0) {
               send chs[prim](semnal, i);
               in[prim].deficit = in[prim].deficit - 1;
            }
          }
          send stop[i](true);
       }}
   }
```

semnal de la Prelucrare: a primit un mesaj

semnal de la Prelucrare: a trimis un mesaj

semnal de confirmare de la alt nod

Prelucrare dorește terminarea

Detecția terminării folosind marcaje

DETECTING TERMINATION OF DISTRIBUTED COMPUTATIONS USING MARKERS

Jayadev Misra
Department of Computer Sciences, University of Texas, Austin, 78712

Abstract

A problem of considerable importance in designing computations by process networks, is detection of termination. We propose a very simple algorithm for termination detection in an arbitrary network using a single marker We show an application of this scheme in solving the problem of token loss detection and token regeneration in a token ring.

Introduction

We study the problem of detecting termination of computation in a network of processes. If every process in a network is *idle*, i.e. waiting for messages in order to carry out further computations, and there are no messages in transit, i.e. all messages that have been sent have been received, then no process will carry out any further computation. It is often important to detect such a situation. Multiphase algorithms [15] in which a phase is to be initiated only upon completion of the previous phase, requires termination detection of a phase. Francez, Rodeh and Sintzoff [11] suggest that it may be easier to devise a distributed algorithm in two steps: (1) design an algorithm that maintains the desired safety properties and eventually guarantees a terminating *global* state; (2) superimpose a termination detection algorithm on the basic algorithm. Deadlock detection [8] which is related to termination detection is of fundamental importance in distributed data bases. Detection of token loss in a token ring can be shown to be a termination detection problem.

We suggest an algorithm for termination detection in an arbitrary network of processes. The algorithm uses a single marker which repeatedly traverses the edges of the network until it detects termination. We make no assumption about the network structure, process behavior or message delays. Our only assumption is that a process receives (only and all) messages from another process in the order sent by the sender. Our solution is symmetric among the processes, i.e. process id's are not used in the solution. As an application of the scheme in this paper, we give a simple and efficient algorithm for detecting token loss and regenerating the token in a token ring network. The marker algorithm has also been applied [14] in *avoiding* deadlocks in distributed simulations.

There has been a considerable amount of work in termination detection. For a specific class of computations, called ***diffusing computations***, Dijkstra and Scholten [2] proposed a very elegant algorithm. Their approach has been extended and applied to a variety of problems [4,5,6,7]. The major drawbacks of their approach are: (1) termination detection algorithm must be initiated whenever the diffusing computation starts and (2) the number of messages used for termination detection is equal to the number of messages in the diffusing computation itself. Our solution does not suffer from these drawbacks.

A series of papers has been published by Francez and co-workers [9,11,12] leading to a marker type algorithm. The algorithm proposed in this paper refines, simplifies and removes some of the restrictions of their approach. Independently Gouda [1] and Dijkstra [3] have proposed similar schemes, again with certain restrictions. All of these schemes use the marker to determine if a process has remained "continuously idle" over an interval of time; the idea of continuous idleness also appears in [8] for deadlock detection.

Recently Chandy and Lamport [10] have proposed a very elegant and general scheme for detecting *stable properties* of a network; a property (proposition) P is stable if it remains true once it becomes true. Clearly properties such as "network computation has terminated," and "a subset of processes are deadlocked" are all stable properties. Even though we treat the narrower problem of termination detection, our solution is simpler and more efficient for this specific problem.

Problem Description

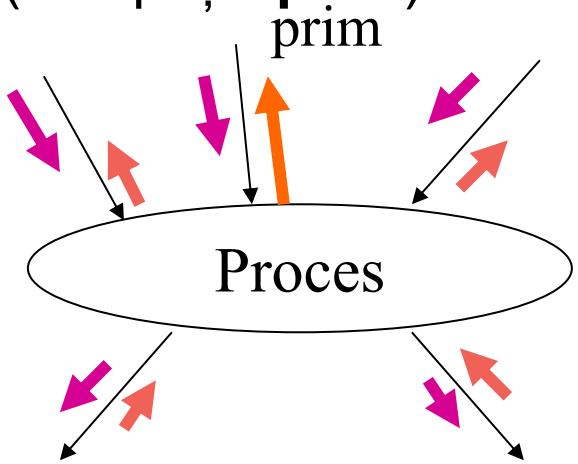
*This work was supported by a grant from IBM.



Misra, Jayadev. "Detecting termination of distributed computations using markers." *Proceedings of the second annual ACM symposium on Principles of distributed computing*. ACM, 1983.

Detecția terminării folosind marcaje (1)

- Procesele transmit mesaje de marcaj la sfârşitului comunicației
- Procesele primesc semnale de confirmare doar pentru marcaje
- Pentru terminare:
  - aşteaptă marcaje pe legăturile de intrare
    - *(să termine cei de la care a primit mesaje)*
  - transmite marcaje pe legăturile de ieşire
    - *(anunță propria terminare celor cărora le-a trimis mesaje)*
  - transmite semnale pe legăturile de intrare (mai puțin **prim**)
    - *(confirmă marcajele primite)*
  - aşteaptă semnale pe legăturile de ieşire
    - *(așteaptă confirmări pentru marcajele trimise)*
  - transmite semnal pentru **prim**



Detecția terminării folosind marcaje (2)

- Pentru fiecare arc se țin două atribute: ***activ*** și ***marcaj.***

- Pentru fiecare **arc de intrare**, se înregistrează dacă:
  - s-a primit un mesaj (se pune ***activ*** pe true);
  - s-a primit un marcaj (se pune ***marcaj*** pe true);
  - s-a transmis un semnal de confirmare (se pun ambele pe false).

- Pentru fiecare **arc de ieşire**, se înregistrează dacă:
  - s-a transmis un mesaj (se pune ***activ*** pe true);
  - s-a transmis un marcaj (se pune ***marcaj*** pe true);
  - s-a primit un semnal de confirmare (se pun ambele pe false).

Detecția terminării folosind marcaje (3)

```
const marcaj = mesaj-special-de-marcaj-sfârșit-date;
typedef enum {semnal, transmitere, recepție,
              recepție-marcaj, terminare} fel;
typedef struct {bool există;
              bool activ;        /*inițial false*/
              bool marcaj} arc;    /*inițial false*/
chan chs[1:N](fel op, int id);
chan chm[1:N](id:int, tip_date date);
chan stop[1:N](bool);
process Nod[i=1 to N]{
  bool term = false;
  arc in[1:N], out[1:N];
  int nsemnale = 0;/*număr semnale așteptate pe out*/
  int prim = 0;
  int nmarcaje;    /*număr marcaje așteptate pe in*/
  int id; fel k;
```

Detecția terminării folosind marcaje (4)

```
while (true) {
    receive chs[i](k, id);
    if (k == recepție) {
      if (prim == 0) prim = id;
      if (NOT in[id].activ) { in[id].activ = true;
                              nmarcaje = nmarcaje + 1;
      }
    else if (k == recepție-marcaj) {
       in[id].marcaj = true; nmarcaje = nmarcaje - 1;
       if (id == prim) term = true; }
    else if (k == transmitere) {
      if (NOT out[id].activ) { out[id].activ = true;
                               nsemnale = nsemnale + 1; }
    else if (k == semnal) {
      out[id].activ = false; out[id].marcaj = false;
      nsemnale = nsemnale - 1;
    }
```

semnal de la Prelucrare: a primit un mesaj

semnal de la Prelucrare: a primit un marcaj

semnal de la Prelucrare: a trimis un mesaj

semnal de confirmare de la alt nod

Detecția terminării folosind marcaje (5)

```
else if (k == terminare) {
      if (NOT term) send stop[i](false);
      else if (term) {
         for [j = 1 to N st (j <> prim AND out[j].activ AND
                             NOT out[j].marcaj)] {
          send chm[j](i, marcaj); out[j].marcaj = true;
         }
         if (nmarcaje <> 0) send stop[i](false);
         else if (nmarcaje == 0) {
             for [j = 1 to n st (in[j].există AND (j<>prim)
                                 AND in[j].activ)] {
                send chs[j](semnal, i);
                in[j].activ = false; in[j].marcaj = false;
             }
             if (nsemnale <> 0) send stop[i](false);
             else if (nsemnale == 0) {
                  if ((i <> sursa) AND (prim <> 0)) {
                     send chs[prim](semnal, i);
                     in[prim].activ = false;
                     in[prim].marcaj = false;
                  }
                  send stop[i](true);
   } } } }
```

Detecția terminării folosind marcaje (6)

```
process Prelucrare[i=1 to N] {
  tip_date data; int id; bool term;
  initializari;
  term = false;
  while (NOT term) {
     receive chm[i](id, data);
     if (data==marcaj) send chs[i](receptie-marcaj,id);
     else if (data<>marcaj) {
          send chs[i](receptie, id);
          realizează prelucrare
          ....
          send chm[j](i, data);
          send chs[i](transmitere, j);
          ...
          if (decizie de terminare) term = true;
     }
  }
  term =false;
  while (NOT term) {
     send chs[i](terminare, i);
     receive stop[i](term);
  }
  stop;
```

Algoritmul lui Huang

Conferences > [1989] Proceedings. The 9th I...

Detecting termination of distributed computations by external agents

Publisher: IEEE

1 Author(s) S.-T. Huang **View All Authors**

34 Paper Citations

408 Full Text Views

Abstract
Authors
References
Citations
Keywords
Metrics

Abstract:
An algorithm is presented that defects for termination of distributed computations by an auxiliary controlling agent. The algorithm assigns a weight W, 0>

Published in: [1989] Proceedings. The 9th International Conference on Distributed Computing Systems

Date of Conference: 5-9 June 1989

Date Added to IEEE *Xplore*: 06 August 2002

Print ISBN: 0-8186-1953-8

INSPEC Accession Number: 3472185

DOI: 10.1109/ICDCS.1989.37933

Publisher: IEEE

Conference Location: Newport Beach, CA, USA, USA

First Page of the Article



Hide First Page Preview

Huang, S-T. "Detecting termination of distributed computations by external agents." *[1989] Proceedings. The 9th International Conference on Distributed Computing Systems*. IEEE, 1989.

Algoritmul lui Huang (1)

- Modelul se bazează pe următoarele:
  - procesele pot fi active sau libere (idle)
  - doar procesele active transmit mesaje
  - procesele libere pot deveni active la recepția unui *mesaj de calcul*
  - procesele active pot deveni libere în orice moment
  - **terminarea**: toate procesele sunt libere şi nu există mesaje de date în tranzit
  - topologia este un graf conex

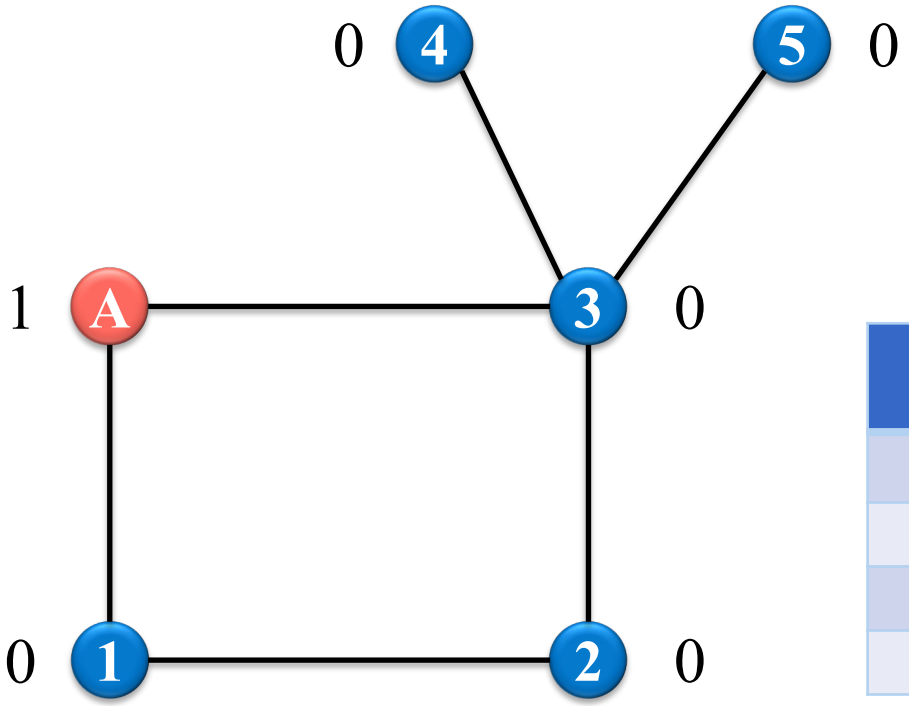
Algoritmul lui Huang (2)

- **Ideea:**
  - Un *agent de control* are inițial ponderea 1.
  - Toate celelalte procese sunt libere și au inițial ponderea 0.
  - Calculul porneşte atunci când *agentul de control* trimite un mesaj de date unui proces.
  - Un proces liber devine activ la recepția unui mesaj de date.
- **Notații:**
  - B(DW)
    - Mesaj de date cu ponderea DW
    - Trimis doar de *agentul de control* sau de un proces activ
  - C(DW)
    - Mesaj de control cu ponderea DW
    - Trimis de un proces activ *agentului de control* când devine liber
  - W
    - Ponderea curentă a unui proces

Algoritmul lui Huang (3)

1. **Send B(DW)**:
   - Găseşte W1, W2 astfel încât W1 > 0, W2 > 0, W1 + W2 = W
   - Pune W = W1 şi transmite B(DW = W2)
2. **Receive B(DW)**:
   - W = W + DW
   - **if** liber -> devine activ **fi**
3. **Send C(DW)**:
   - send C(DW = W) *agentului de control*
   - devine liber
4. **Receive C(DW) (doar *agentul de control*)**:
   - W = W + DW
   - **if** W = 1 -> declară “terminarea” **fi**

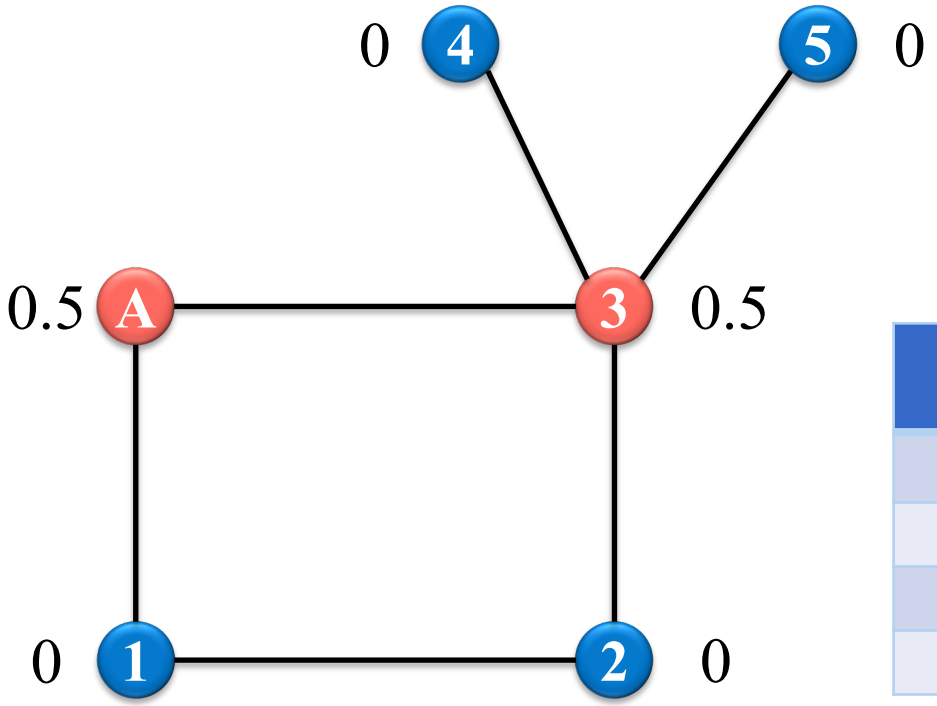
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
A	3	B(DW=0.5)

TIMP
0

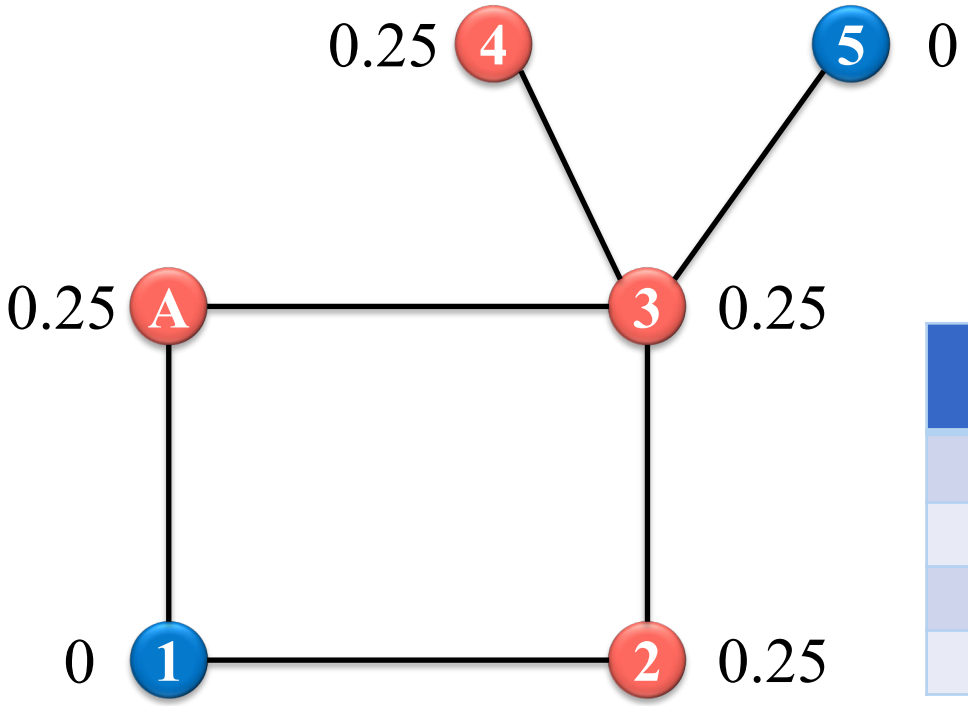
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
A	2	B(DW=0.25)
3	4	B(DW=0.25)

TIMP
1

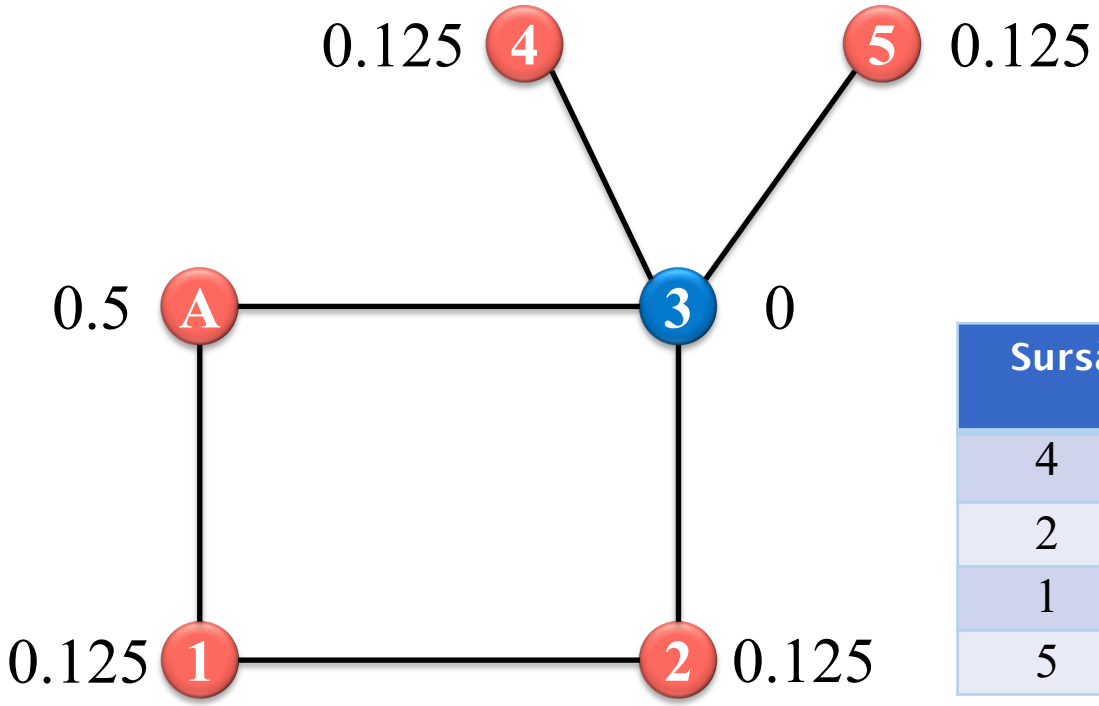
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
4	1	B(DW=0.125)
3	A	C(DW=0.25)
2	5	B(DW=0.125)

TIMP
2

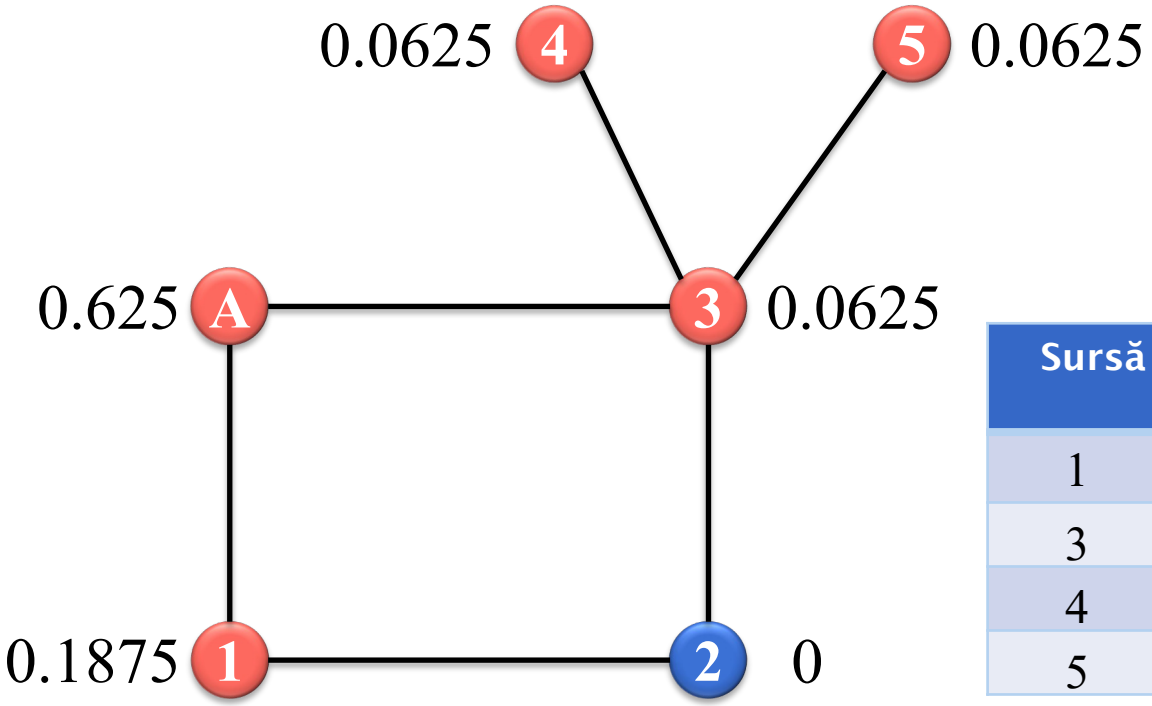
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
4	1	B(DW=0.0625)
2	A	C(DW=0.125)
1	3	B(DW=0.0625)
5	1	B(DW=0.0625)

TIMP
3

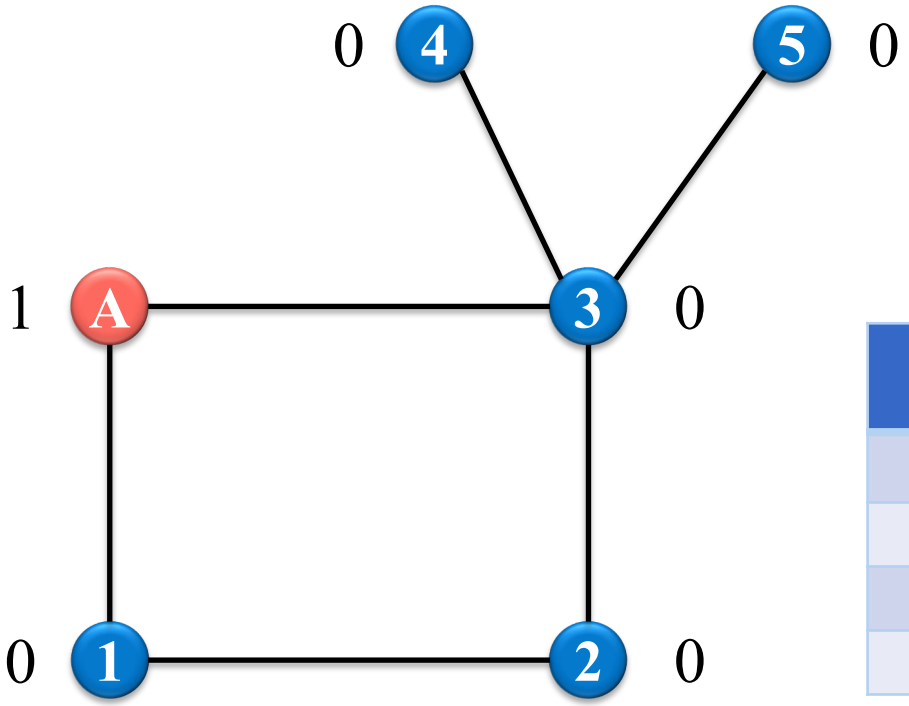
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
1	A	C(DW=0.1875)
3	A	C(DW=0.0625)
4	A	C(DW=0.0625)
5	A	C(DW=0.0625)

TIMP
4

Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj

TIMP
5

Sumar

- Problematica terminării programelor distribuite
- Cazul proceselor organizate în inel
- Tehnica jetoanelor pentru cazul general
- Confirmarea mesajelor
- Algoritmul Dijkstra-Scholten
- Detecția terminării folosind marcaje
- Algoritmul lui Huang

Final de semestru

- Tema 2
  - deadline hard mâine (12.02)
- Tema 3
  - deadline soft duminică (15.01)
  - deadline hard marți (17.01)
- Test practic
  - săptămâna viitoare la laborator
- Feedback
  - deschis până vinerea viitoare (20.01)
- Curs
  - un ultim curs miercurea viitoare (18.01)